



(Company No. 101067-P)

الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يُونِيسَيْتِي إِسْلَامُ أَنْتَارَا بَغْسَا مِلْدِسِيَا

Garden of Knowledge and Virtue

KULLIYAH OF INFORMATION AND COMMUNICATION TECHNOLOGY

SEM 2 2022/2023

CSCI 2304 - INTELLIGENT SYSTEMS

SECTION 2

Report - Group Project

Title: Phishing URL Detection Using KNN Model

GROUP: DITTO

LECTURER:

ASSOC. PROF. TS. DR. AMELIA RITAHANI BINTI ISMAIL

NAME	MATRIC NO.
NOR ANIS FAZLIN BINTI MOHAMAD ADIB	2014954
SORFINA ALYIA BINTI JAZRRY	2017326

Presentation Link: [Phishing URL Detection](#)

Table of Contents

1. Abstract	2
2. Introduction	3
3. AI Solution/Technique	6
3.1: Literature Review	6
3.2: Code	6
4. Result	22
5. Deployment	24
6. Conclusion	25
7. Reference	27

1. Abstract

Phishing attacks pose a significant threat in today's digital landscape, exploiting fake websites to deceive users into revealing sensitive information. This report examines techniques for detecting phishing URLs based on the Phishing Websites Features study by Muhammad et al. (2015). By employing supervised machine learning, we aim to automate the process of identifying and predicting the legitimacy of URLs. Utilizing a dataset with over 11,000 rows from Kaggle, we analyze 30 features and normalize the data for classification purposes. Through the implementation of the K-Nearest Neighbor algorithm and a Streamlit GUI, accurate results are obtained. The collaborative effort delves into the distinct characteristics of phishing URLs and how they can be differentiated from legitimate ones. Users are urged to exercise caution when interacting with links and sharing personal information. Notable features such as the presence of "@" symbols, the use of "://" for redirection, favicon analysis, domain registration length, and non-standard port usage contribute to identifying phishing URLs. To further improve detection, future endeavors may explore the implementation of behavioral analysis techniques. Real-time monitoring and analysis of user behavior can significantly enhance the capability of phishing URL detection systems. This assessment expands our understanding of phishing URLs and the model used and provides insights into their differentiation from legitimate URLs. The importance of user vigilance combined with advancements in detection techniques contributes to a safer online environment.

2. Introduction

Phishing attacks are becoming increasingly prevalent in today's rapidly developing digital scene. Phishing is a kind of online fraud that includes the use of fake websites that seem like authentic ones in order to fraud users into disclosing personal information such as usernames, passwords, and credit card information. The analysis of website URLs is one of the most common techniques for detecting fraud attempts. This report examines the various techniques and strategies employed in URL-based fraud detection, discussed from [Phishing Websites Features](#), a detailed study by Muhammad et. al. (2015). This paper entails the characteristics that distinguish phishing URLs from legitimate ones, as well as the machine learning techniques that can be used to automate this detection process.

The objective of this report is to analyze the unique characteristics of phishing URLs, and use the supervised machine learning used to implement the features discussed in the paper to automate the phishing detector system by inputting the URL. The dataset consists of 30 features columns which is also discussed in the paper mentioned above:

Column Name	Description	Attribution	Parameter
UsingIP	Indicates whether the URL is using an IP address instead of a domain name	Potentially relevant, as phishing URLs may use IP addresses to deceive users	-1: Phishing, 1: Legitimate
LongURL	Indicates whether the URL is a long URL	Length of a URL could be indicative of suspicious activity, potentially informative	-1: Phishing, 0: Suspicious, 1: Legitimate
ShortURL	Indicates whether the URL is a short URL	Shortened URLs are commonly used in phishing, potentially informative	-1: Phishing, 1: Legitimate
Symbol@	Indicates the presence of the symbol "@" in the URL	"@" symbol is not commonly used in regular URLs, presence might indicate suspicious URLs	-1: Phishing, 1: Legitimate
Redirecting//	Indicates whether the URL contains the substring "Redirecting//"	Presence of redirecting mechanisms can be exploited by attackers, potentially informative	The position of the most recent "/" in the URL > 7: -1: Phishing, Otherwise: 1: Legitimate
PrefixSuffix-	Indicates the presence of a prefix or suffix in the URL	Additional prefixes or suffixes might indicate suspicious or mimicry behavior, potentially informative	-1: Phishing, 1: Legitimate
SubDomains	Number of subdomains in the URL	Higher number of subdomains might indicate suspicious or malicious behavior, potentially informative	More than 2 dots in the domain: -1: Phishing, 2 dots in the domain: 0: Suspicious, 1 dot in the domain: 1: Legitimate

HTTPS	Indicates whether the URL uses HTTPS (secure HTTP)	Presence of HTTPS is a strong indicator of a legitimate website, likely to be important	Other than the below: -1: Phishing, Using HTTPS but issuer is not trusted: 0: Suspicious, Using HTTPS, trusted issuer, and certificate age ≥ 1 year: 1: Legitimate
DomainRegLen	Length of the domain registration	Phishing websites are often newly created with shorter domain registration lengths, potentially informative	-1: Phishing, 1: Legitimate
Favicon	Indicates the presence of a favicon (website icon) in the URL	Phishing websites may not have proper favicons, absence can be a red flag, potentially informative	-1: Phishing, 1: Legitimate
NonStdPort	Indicates whether the URL uses a non-standard port number	Presence of a non-standard port could indicate suspicious activity, potentially informative	-1: Phishing, 1: Legitimate
HTTPSDomainURL	Indicates whether the domain name in the URL is present in the HTTPS certificate	Matching domain name in HTTPS certificate is indicative of a legitimate website, likely important	-1: Phishing, 1: Legitimate
RequestURL	Indicates the presence of a request URL in the URL string	Additional request URLs can be indicative of suspicious activity, potentially informative	-1: Phishing, 1: Legitimate
AnchorURL	Indicates whether the URL contains an anchor URL	Anchor URLs can be used for redirection to malicious websites, potentially informative	Other than the below: -1: Phishing, % of URL of Anchor $\geq 31\%$ and $\leq 67\%$: 0: Suspicious, % of URL of Anchor $< 31\%$: 1: Legitimate
LinksInScriptTags	Number of links within script tags in the URL	Presence of multiple links within script tags, specifically <Meta>, <Script> and <Link> tags can indicate suspicious activity	1: Legitimate, -1: Phishing, 0: Suspicious
ServerFormHandler	Presence of a server form handler in the URL	Server form handlers can be indicative of data collection by attackers, potentially informative	1: Legitimate, -1: Phishing, 0: Suspicious
InfoEmail	Presence of uses of "mail()" or "mailto:" function	Email addresses in URLs can be the phisher's personal email, used to submit the user information.	-1: Phishing, 1: Legitimate
AbnormalURL	Indicates whether the URL is considered abnormal based on specific characteristics	Abnormal URLs like the host name is not included in the URL may exhibit suspicious patterns.	-1: Phishing, 1: Legitimate
WebsiteForwarding	Indicates whether the website is using forwarding techniques	Forwarding techniques can be used to redirect users to malicious websites, potentially informative	0: Suspicious, 1: Legitimate

StatusBarCust	Presence of custom status bar in the URL	Custom status bars may be used to deceive users, potentially informative	1: Legitimate, -1: Phishing
DisableRightClick	Indicates whether right-click functionality is disabled on the website	Disabling right-click can be used to restrict user actions, potentially informative	1: Legitimate, -1: Phishing
UsingPopupWindow	Indicates the use of popup windows in the URL	Popup windows with text fields can be used for malicious purposes, potentially informative	1: Legitimate, -1: Phishing
IframeRedirection	Indicates whether the URL uses iframe redirection	Iframe redirection can be used to redirect users to malicious websites, potentially informative	1: Legitimate, -1: Phishing
AgeofDomain	Age of the domain registration	Phishing websites are often newly created, shorter age might be indicative of suspicious URLs	-1: Phishing, 1: Legitimate
DNSRecording	Indicates whether DNS records exist for the domain	Absence of DNS records might indicate suspicious URLs, potentially informative	-1: Phishing, 1: Legitimate
WebsiteTraffic	Estimated website traffic	Higher traffic might indicate legitimate websites, potentially informative	-1: Phishing, 0: Suspicious, 1: Legitimate
PageRank	PageRank score of the website	Higher PageRank scores might indicate legitimate websites, potentially informative	-1: Phishing, 1: Legitimate
GoogleIndex	Indicates whether the website is indexed by Google	Indexed websites are likely legitimate, potentially informative	-1: Phishing, 1: Legitimate
LinksPointingToPage	Number of external links pointing to the page	Higher number of external links might indicate legitimate websites, potentially informative	-1: Phishing, 0: Suspicious, 1: Legitimate
StatsReport	Indicates whether website statistics are available	Availability of website statistics might indicate legitimate websites, potentially informative	-1: Phishing, 1: Legitimate

In this report, we employ supervised machine learning techniques, specifically the K-Nearest Neighbors (KNN) model. This model was trained using our dataset, enabling them to classify URLs as either phishing or legitimate based on the extracted features. The report further explores into the performance analysis of these classification models and implements it to a phishing URL detector application. Finally, we discuss the strengths, weaknesses, and potential improvement areas of the KNN model to the system.

3. AI Solution/Technique

3.1: Literature Review

The K-Nearest Neighbors (KNN) algorithm provides several distinct advantages in detecting phishing websites using URL data. Numerous studies have demonstrated that KNN offers high accuracy, robustness to noisy data, low computational complexity, and feature selection capabilities when compared to other models. In a recent study by Shahrivari V. et. al. (2020), the authors discussed how KNN's ability to classify websites based on similarity to known phishing or legitimate examples contributes significantly to its accuracy in identifying potential threats. As explored in a study conducted by Taha (2017), KNN's robustness allows it to handle noisy data effectively, making it reliable for detecting phishing websites amidst data imperfections. Furthermore, its low computational complexity facilitates efficient processing of large datasets, enabling quick identification of potentially malicious URLs. By leveraging fundamental visual features and similarity-based classification, KNN effectively distinguishes between legitimate and malicious websites and enhances its efficacy in combating phishing attacks, as discussed thoroughly by Safi et. al. (2023). Overall, the inherent advantages of KNN position it as a valuable and versatile tool in the ongoing battle against the ever-evolving threats posed by phishing attacks.

3.2: Code

We utilized the K-Nearest Neighbors (KNN) model, which was trained and evaluated, and implemented it into an application using Streamlit, an open-source app framework designed for Machine Learning and Data Science.

```
import os
import streamlit as st
import bs4
import re
import datetime
import numpy as np
import pandas as pd
import seaborn as sns
import random
import socket
import time
import urllib.request
import requests
```

```

import whois
import ipaddress
import joblib
import google

from dateutil.parser import parse as date_parse
from sklearn import metrics
from urllib.parse import urlparse

data = pd.read_csv('phishing.csv')

# Load and preprocess the data
st.info("Preprocess the data and build model")
X = data.drop(["class", "Index"], axis=1)
y = data["class"]

# Split the data into training and test sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify = y)

# Train the model
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=1)

# Train the model
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=1)

from sklearn.model_selection import cross_val_score
cv_scores = cross_val_score(knn, X, y, cv=10)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
# Calculate accuracy score
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)

# Save the model
joblib.dump(knn, 'phishingurl_knn.pkl')

class FeatureExtraction:

```



```

features = []
def __init__(self,url):
    self.features = []
    self.url = url
    self.domain = ""
    self.whois_response = ""
    self.urlparse = ""
    self.response = ""
    self.soup = ""

    try:
        self.response = requests.get(url)
        self.soup = BeautifulSoup(response.text, 'html.parser')
    except:
        pass

    try:
        self.urlparse = urlparse(url)
        self.domain = self.urlparse.netloc
    except:
        pass

    try:
        self.whois_response = whois.whois(self.domain)
    except:
        pass

    self.features.append(self.UsingIp())
    self.features.append(self.longUrl())
    self.features.append(self.shortUrl())
    self.features.append(self.symbol())
    self.features.append(self.redirecting())
    self.features.append(self.prefixSuffix())
    self.features.append(self.SubDomains())
    self.features.append(self.Hppts())
    self.features.append(self.DomainRegLen())
    self.features.append(self.Favicon())

    self.features.append(self.NonStdPort())
    self.features.append(self.HTTPSDomainURL())
    self.features.append(self.RequestURL())

```

```

self.features.append(self.AnchorURL())
self.features.append(self.LinksInScriptTags())
self.features.append(self.ServerFormHandler())
self.features.append(self.InfoEmail())
self.features.append(self.AbnormalURL())
self.features.append(self.WebsiteForwarding())
self.features.append(self.StatusBarCust())

self.features.append(self.DisableRightClick())
self.features.append(self.UsingPopupWindow())
self.features.append(self.IframeRedirection())
self.features.append(self.AgeofDomain())
self.features.append(self.DNSRecording())
self.features.append(self.WebsiteTraffic())
self.features.append(self.PageRank())
self.features.append(self.GoogleIndex())
self.features.append(self.LinksPointingToPage())
self.features.append(self.StatsReport())

# 1.UsingIp
def UsingIp(self):
    # st.success("1 - UsingIP")
    try:
        ipaddress.ip_address(self.url)
        return -1
    except:
        return 1

# 2.longUrl
def longUrl(self):
    # st.success("2 - longUrl")
    if len(self.url) < 54:
        return 1
    if len(self.url) >= 54 and len(self.url) <= 75:
        return 0
    return -1

# 3.shortUrl
def shortUrl(self):
    # st.success("3 - ShortURL")
    match = re.search('bit\.ly|goo\.gl|shorte\.st|go2l\.ink|x\.co|ow\.ly|t\.co|tinyurl|tr\.im|is\.gd|cli\.gs|'

```

```

'yfrog\.com|migre\.me|ff\.im|tiny\.cc|url4\.eu|twit\.ac|su\.pr|twurl\.nl|snipurl\.com|'
'short\.to|BudURL\.com|ping\.fm|post\.ly|Just\.as|bkite\.com|snipr\.com|fic\.kr|loopt\.us|'
'doiop\.com|short\.ie|kl\.am|wp\.me|rubyurl\.com|om\.ly|to\.ly|bit\.do|t\.co|lnkd\.in|'
'db\.tt|qr\.ae|adf\.ly|goo\.gl|bitly\.com|cur\.lv|tinyurl\.com|ow\.ly|bit\.ly|ity\.im|'
'q\.gs|is\.gd|po\.st|bc\.vc|twitthis\.com|u\.to|j\.mp|buzurl\.com|cutt\.us|u\.bb|yourls\.org|'
'x\.co|prettylinkpro\.com|scrnch\.me|filoops\.info|vzturl\.com|qr\.net|1url\.com|tweez\.me|v\.gd|tr\.im|link\.zip\.net',
self.url)

    if match:
        return -1
    return 1

# 4.Symbol@
def symbol(self):
    # st.success("4 - Symbol@")
    if re.findall("@",self.url):
        return -1
    return 1

# 5.Redirecting//
def redirecting(self):
    # st.success("5 - Redirecting")
    if self.url.rfind("/")>6:
        return -1
    return 1

# 6.prefixSuffix
def prefixSuffix(self):
    # st.success("6 - prefixSuffix")
    try:
        match = re.findall('-', self.domain)
        if match:
            return -1
        return 1
    except:
        return -1

# 7.SubDomains
def SubDomains(self):
    # st.success("7 - SubDomains")
    dot_count = len(re.findall(".", self.url))
    if dot_count == 1:
        return 1

```

```

elif dot_count == 2:
    return 0
return -1

# 8.HTTPS
def Hppts(self):
    # st.success("8 - HTTPS")
    try:
        https = self.urlparse.scheme
        if 'https' in https:
            return 1
        return -1
    except:
        return 1

# 9.DomainRegLen
def DomainRegLen(self):
    # st.success("9 - DomainRegLen")
    try:
        expiration_date = self.whois_response.expiration_date
        creation_date = self.whois_response.creation_date
        try:
            if(len(expiration_date)):
                expiration_date = expiration_date[0]
        except:
            pass
        try:
            if(len(creation_date)):
                creation_date = creation_date[0]
        except:
            pass

        age = (expiration_date.year-creation_date.year)*12+ (expiration_date.month-creation_date.month)
        if age >=12:
            return 1
        return -1
    except:
        return -1

# 10. Favicon
def Favicon(self):
    # st.success("10 - Favicon")

```

```

try:
    for head in self.soup.find_all('head'):
        for head.link in self.soup.find_all('link', href=True):
            dots = [x.start(0) for x in re.finditer('\.', head.link['href'])]
            if self.url in head.link['href'] or len(dots) == 1 or domain in head.link['href']:
                return 1
    return -1
except:
    return -1

# 11. NonStdPort
def NonStdPort(self):
    # st.success("11 - NonStdPort")
    try:
        port = self.domain.split(":")
        if len(port)>1:
            return -1
        return 1
    except:
        return -1

# 12. HTTPSDomainURL
def HTTPSDomainURL(self):
    # st.success("12 - HTTPSDomainURL")
    try:
        if 'https' in self.domain:
            return -1
        return 1
    except:
        return -1

# 13. RequestURL
def RequestURL(self):
    # st.success("13 - RequestURL")
    try:
        for img in self.soup.find_all('img', src=True):
            dots = [x.start(0) for x in re.finditer('\.', img['src'])]
            if self.url in img['src'] or self.domain in img['src'] or len(dots) == 1:
                success = success + 1
            i = i+1

        for audio in self.soup.find_all('audio', src=True):

```

```

        dots = [x.start(0) for x in re.finditer('\.', audio['src'])]
        if self.url in audio['src'] or self.domain in audio['src'] or len(dots) == 1:
            success = success + 1
        i = i+1

    for embed in self.soup.find_all('embed', src=True):
        dots = [x.start(0) for x in re.finditer('\.', embed['src'])]
        if self.url in embed['src'] or self.domain in embed['src'] or len(dots) == 1:
            success = success + 1
        i = i+1

    for iframe in self.soup.find_all('iframe', src=True):
        dots = [x.start(0) for x in re.finditer('\.', iframe['src'])]
        if self.url in iframe['src'] or self.domain in iframe['src'] or len(dots) == 1:
            success = success + 1
        i = i+1

    try:
        percentage = success/float(i) * 100
        if percentage < 22.0:
            return 1
        elif((percentage >= 22.0) and (percentage < 61.0)):
            return 0
        else:
            return -1
    except:
        return 0
    except:
        return -1

# 14. AnchorURL
def AnchorURL(self):
    # st.success("14 - AnchorURL")
    try:
        i,unsafe = 0,0
        for a in self.soup.find_all('a', href=True):
            if "#" in a['href'] or "javascript" in a['href'].lower() or "mailto" in a['href'].lower() or not (url in a['href'] or self.domain in a['href']):
                unsafe = unsafe + 1
            i = i + 1

    try:

```

```

percentage = unsafe / float(i) * 100
if percentage < 31.0:
    return 1
elif ((percentage >= 31.0) and (percentage < 67.0)):
    return 0
else:
    return -1
except:
    return -1
except:
    return -1

# 15. LinksInScriptTags
def LinksInScriptTags(self):
    # st.success("15 - LinksInScriptTags")
    try:
        i, success = 0, 0

        for link in self.soup.find_all('link', href=True):
            dots = [x.start(0) for x in re.finditer('\.', link['href'])]
            if self.url in link['href'] or self.domain in link['href'] or len(dots) == 1:
                success = success + 1
            i = i + 1

        for script in self.soup.find_all('script', src=True):
            dots = [x.start(0) for x in re.finditer('\.', script['src'])]
            if self.url in script['src'] or self.domain in script['src'] or len(dots) == 1:
                success = success + 1
            i = i + 1

        try:
            percentage = success / float(i) * 100
            if percentage < 17.0:
                return 1
            elif ((percentage >= 17.0) and (percentage < 81.0)):
                return 0
            else:
                return -1
        except:
            return 0
        except:
            return -1

```

```

# 16. ServerFormHandler
def ServerFormHandler(self):
    # st.success("16 - ServerFormHandler")
    try:
        if len(self.soup.find_all('form', action=True))==0:
            return 1
        else :
            for form in self.soup.find_all('form', action=True):
                if form['action'] == "" or form['action'] == "about:blank":
                    return -1
                elif self.url not in form['action'] and self.domain not in form['action']:
                    return 0
                else:
                    return 1
    except:
        return -1

# 17. InfoEmail
def InfoEmail(self):
    # st.success("17 - InfoEmail")
    try:
        if re.findall(r"[mail\(\)]mailto:?", self.soup):
            return -1
        else:
            return 1
    except:
        return -1

# 18. AbnormalURL
def AbnormalURL(self):
    # st.success("18 - AbnormalURL")
    try:
        if self.response.text == self.whois_response:
            return 1
        else:
            return -1
    except:
        return -1

# 19. WebsiteForwarding
def WebsiteForwarding(self):

```



```

# st.success("19 - WebsiteForwarding")
try:
    if len(self.response.history) <= 1:
        return 1
    elif len(self.response.history) <= 4:
        return 0
    else:
        return -1
except:
    return -1

# 20. StatusBarCust
def StatusBarCust(self):
    # st.success("20 - StatusBarCust")
    try:
        if re.findall("<script>.+onmouseover.+</script>", self.response.text):
            return 1
        else:
            return -1
    except:
        return -1

# 21. DisableRightClick
def DisableRightClick(self):
    # st.success("21 - DisableRightClick")
    try:
        if re.findall(r"event.button ?== ?2", self.response.text):
            return 1
        else:
            return -1
    except:
        return -1

# 22. UsingPopupWindow
def UsingPopupWindow(self):
    # st.success("22 - UsingPopupWindow")
    try:
        if re.findall(r"alert\(", self.response.text):
            return 1
        else:
            return -1
    except:

```

```

        return -1

# 23. IframeRedirection
def IframeRedirection(self):
    val = 0
    try:
        # if re.findall(r"<iframe>|<frameBorder>]", self.response.text):
        #     return 1
        # else:
        #     return -1
        val = 1 if re.findall(r"<iframe>|<frameBorder>]", self.response.text) else -1; return val
    st.success("23 - IframeRedirection : " + str(val))
    return val
except:
    return -1

# 24. AgeofDomain
def AgeofDomain(self):
    # st.success("24 - AgeofDomain")
    try:
        creation_date = self.whois_response.creation_date
        try:
            if(len(creation_date)):
                creation_date = creation_date[0]
        except:
            pass

        today = date.today()
        age = (today.year-creation_date.year)*12+(today.month-creation_date.month)
        if age >=6:
            return 1
        return -1
    except:
        return -1

# 25. DNSRecording
def DNSRecording(self):
    # st.success("25 - DNSRecording")
    try:
        creation_date = self.whois_response.creation_date
        try:

```

```

        if(len(creation_date)):
            creation_date = creation_date[0]
    except:
        pass

    today = date.today()
    age = (today.year-creation_date.year)*12+(today.month-creation_date.month)
    if age >=6:
        return 1
    return -1
except:
    return -1

# 26. WebsiteTraffic
def WebsiteTraffic(self):
    # st.success("26 - WebsiteTraffic ")
    try:
        rank = BeautifulSoup(urllib.request.urlopen("http://data.alexa.com/data?cli=10&dat=s&url=" + url).read(),
"xml").find("REACH")["RANK"]
        if (int(rank) < 100000):
            return 1
        return 0
    except :
        return -1

# 27. PageRank
def PageRank(self):
    # st.success("27 - PageRank")
    try:
        prank_checker_response = requests.post("https://www.checkpagerank.net/index.php", {"name": self.domain})

        global_rank = int(re.findall(r"Global Rank: ([0-9]+)", rank_checker_response.text)[0])
        if global_rank > 0 and global_rank < 100000:
            return 1
        return -1
    except:
        return -1

# 28. GoogleIndex
def GoogleIndex(self):
    # st.success("28 - GoogleIndex")

```

```

try:
    site = search(self.url, 5)
    if site:
        return 1
    else:
        return -1
except:
    return 1

# 29. LinksPointingToPage
def LinksPointingToPage(self):
    # st.success("29. LinksPointingToPage")
    try:
        number_of_links = len(re.findall(r"<a href=", self.response.text))
        if number_of_links == 0:
            return 1
        elif number_of_links <= 2:
            return 0
        else:
            return -1
    except:
        return -1

# 30. StatsReport
def StatsReport(self):
    # st.success("30 - StatsReport")
    try:
        url_match = re.search(
            'at\ua|usa\cc|baltazarpresntes\.com\.br|pe\.hu|esy\.es|hol\.es|sweddy\.com|myjino\.ru|96\.lt|ow\.ly', url)
        ip_address = socket.gethostbyname(self.domain)
        ip_match =
re.search('146\.112\.61\.108|213\.174\.157\.151|121\.50\.168\.88|192\.185\.217\.116|78\.46\.211\.158|181\.174\.165\.13|46\.242\.1
45\.103|121\.50\.168\.40|83\.125\.22\.219|46\.242\.145\.98|'

'107\.151\.148\.44|107\.151\.148\.107|64\.70\.19\.203|199\.184\.144\.27|107\.151\.148\.108|107\.151\.148\.109|119\.28\.52\.61|54\
.83\.43\.69|52\.69\.166\.231|216\.58\.192\.225|'

'118\.184\.25\.86|67\.208\.74\.71|23\.253\.126\.58|104\.239\.157\.210|175\.126\.123\.219|141\.8\.224\.221|10\.10\.10\.10|43\.229\
.108\.32|103\.232\.215\.140|69\.172\.201\.153|'

'216\.218\.185\.162|54\.225\.104\.146|103\.243\.24\.98|199\.59\.243\.120|31\.170\.160\.61|213\.19\.128\.77|62\.113\.226\.131|208\
.100\.26\.234|195\.16\.127\.102|195\.16\.127\.157|'

```

```

'34\196\13\28|103\224\212\222|172\217\4\225|54\72\9\51|192\64\147\141|198\200\56\183|23\253\164\103|52\48\1
91\26|52\214\197\72|87\98\255\18|209\99\17\27|'

'216\38\62\18|104\130\124\96|47\89\58\141|78\46\211\158|54\86\225\156|54\82\156\19|37\157\192\102|204\11\56\
48|110\34\231\42', ip_address)

    if url_match:
        return -1
    elif ip_match:
        return -1
    return 1
except:
    return 1

def getFeaturesList(self):
    return self.features

custom_columns = ['UsingIP', 'LongURL', 'ShortURL', 'Symbol@', 'Redirecting/',
                  'PrefixSuffix-', 'SubDomains', 'HTTPS', 'DomainRegLen', 'Favicon',
                  'NonStdPort', 'HTTPSDomainURL', 'RequestURL', 'AnchorURL',
                  'LinksInScriptTags', 'ServerFormHandler', 'InfoEmail', 'AbnormalURL',
                  'WebsiteForwarding', 'StatusBarCust', 'DisableRightClick',
                  'UsingPopupWindow', 'IframeRedirection', 'AgeofDomain', 'DNSRecording',
                  'WebsiteTraffic', 'PageRank', 'GoogleIndex', 'LinksPointingToPage',
                  'StatsReport']

st.title("URL Phishing Detection")
# Initialize the random URL
random_url = ""

url_input = st.text_input("Enter the URL", random_url)

# Process the URL on form submission
if st.button("Submit"):
    st.info("Processing " + url_input)
    obj = FeatureExtraction(url_input)
    x = np.array(obj.getFeaturesList()).reshape(1, 30)

    # Display the DataFrame as a table in Streamlit
    df = pd.DataFrame(x, columns=custom_columns)
    st.table(df.T) #Transpose the table rows and columns

```

```
#for one output for the class prediction
pred = knn.predict(x)
if pred == 1:
    st.write("Verdict : Legitimate website detected")
else:
    st.write("Verdict : Caution! Suspicious website detected")

st.write("Model accuracy:", accuracy)
```

4. Result

```
▼ KNeighborsClassifier
KNeighborsClassifier(n_neighbors=1)
```

For k, we use 1 as it is the most optimal number

```
Confusion Matrix:
[[1388   81]
 [  59 1789]]
```

The confusion matrix reveals that there were 1388 true positives, 81 false positives, 59 false negatives, and 1789 true negatives in the classification task.

```
K-Nearest Neighbors : Accuracy on training Data: 0.989
K-Nearest Neighbors : Accuracy on test Data: 0.958
```

```
K-Nearest Neighbors : f1_score on training Data: 0.990
K-Nearest Neighbors : f1_score on test Data: 0.962
```

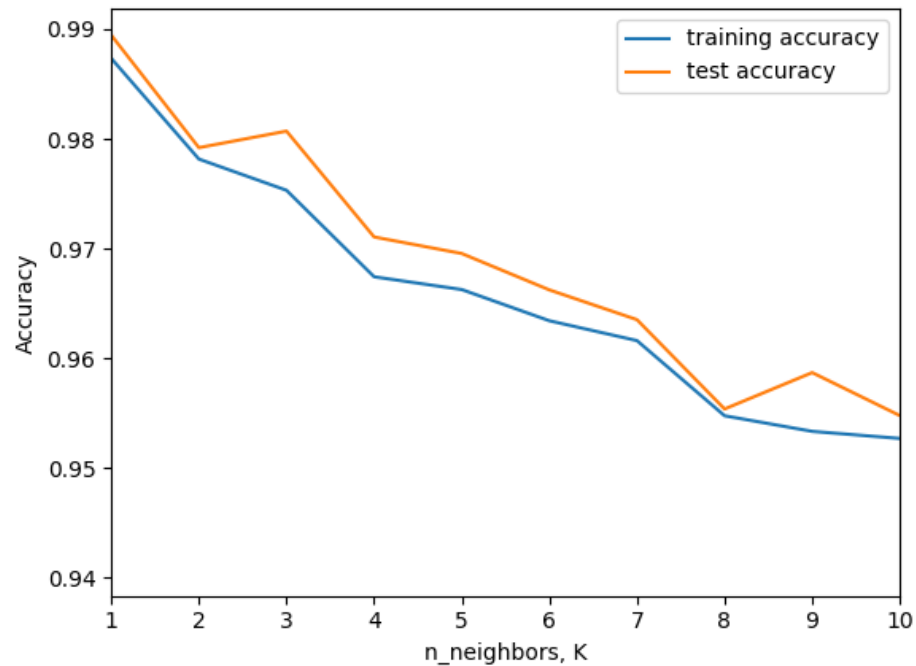
```
K-Nearest Neighbors : Recall on training Data: 0.992
K-Nearest Neighbors : Recall on test Data: 0.968
```

```
K-Nearest Neighbors : precision on training Data: 0.988
K-Nearest Neighbors : precision on test Data: 0.957
```

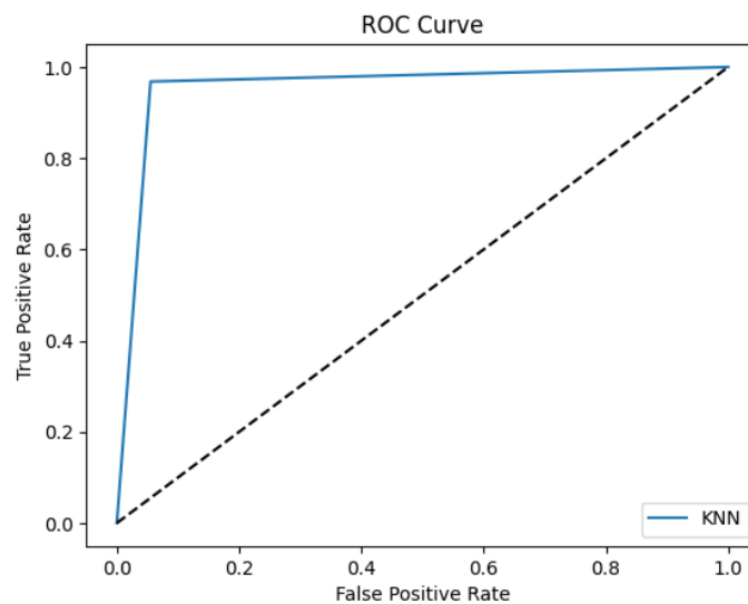
Based on the picture above, the K-Nearest Neighbors model achieved an accuracy of 0.989 on the training data and an accuracy of 0.958 on the test data, indicating its overall performance. Furthermore, it obtained an f1_score of 0.990 on the training data and 0.962 on the test data, which measures the balance between precision and recall. A high recall value of 0.992 on the training data and 0.968 on the test data was displayed by the model, demonstrating its accuracy in identifying positive events. Also, the model correctly labeled positive predictions with a precision of 0.988 on the training data and a precision of 0.957 on the test data.

	precision	recall	f1-score	support
-1	0.96	0.94	0.95	1469
1	0.96	0.97	0.96	1848
accuracy			0.96	3317
macro avg	0.96	0.96	0.96	3317
weighted avg	0.96	0.96	0.96	3317

Based on the picture above, the K-Nearest Neighbors model achieved high precision (0.96) and recall (0.94) for the negative class, resulting in a balanced f1-score (0.95). For the positive class, the model had a precision of 0.96, recall of 0.97, and an f1-score of 0.96.



Based on the graph above, the most optimal number of k is 1 which results with the highest training and testing accuracy.



As the threshold changed, the TPR and FPR values were adjusted accordingly, resulting in different points on the ROC curve. The area under the curve (AUC) which is 0.95647 represents the overall performance of the model, with a higher value indicating better classification accuracy.

5. Deployment

The application provides a user-friendly interface where users can input a website URL. The trained ML model then generates an output by classifying the website as either legitimate or a phishing website. The output of classification includes all 30 columns, which correspond to the 30 previously discussed features. The website's characteristics are presented in a comprehensive manner through individual classification of each feature in a table format.

Preprocess the data and build model

URL Phishing Detection

Enter the URL

Processing <http://p7hpqzqyo.zsfugi.net/>

	0
UsingIP	1
LongURL	1
ShortURL	1
Symbol@	1
Redirecting//	1
PrefixSuffix	1
SubDomains	0
HTTPS	-1
DomainRegLen	-1
Favicon	-1
NonStdPort	1
HTTPSDomainURL	1
RequestURL	-1
AnchorURL	-1
LinkinScriptTags	-1
ServerFormHandler	-1
InfoEmail	-1
AbnormalURL	-1
WebsiteForwarding	-1
StatusBarCust	-1
DisableRightClick	-1
UsingPopupWindow	-1
IframeRedirection	-1
AgeofDomain	-1
DNSRecording	-1
WebsiteTraffic	-1
PageRank	-1
GoogleIndex	1
LinksPointingToPage	-1
StatsReport	1

Verdict : Caution! Suspicious website detected

Model accuracy: 0.9577931866144186

Preprocess the data and build model

URL Phishing Detection

Enter the URL

Processing <https://www.alrasia.com>

	0
UsingIP	1
LongURL	1
ShortURL	1
Symbol@	1
Redirecting//	1
PrefixSuffix	1
SubDomains	0
HTTPS	1
DomainRegLen	1
Favicon	-1
NonStdPort	1
HTTPSDomainURL	1
RequestURL	-1
AnchorURL	-1
LinkinScriptTags	-1
ServerFormHandler	-1
InfoEmail	-1
AbnormalURL	-1
WebsiteForwarding	1
StatusBarCust	-1
DisableRightClick	-1
UsingPopupWindow	-1
IframeRedirection	1
AgeofDomain	-1
DNSRecording	-1
WebsiteTraffic	-1
PageRank	-1
GoogleIndex	1
LinksPointingToPage	1
StatsReport	1

Verdict : Legitimate website detected

Model accuracy: 0.9577931866144186

The above is the output which showcases both a suspicious phishing website and a legitimate website.

6. Conclusion

Phishing URL has been around since decades ago along with the advancement of technology. The bad guy keeps thinking of a way to scam or lie to the gullible people in order to steal their money or private information. In this project, we managed to find a dataset that collected more than 11 thousands links combining phishing and legitimate links on Kaggle. The datasets have 30 features that have proved to be sound and effective in predicting phishing websites. All the data have been normalized into 1, 0 or -1 where 1 is legitimate, 0 is suspicious (not all features have suspicious) and -1 which is phishing. The result of the code displays the accuracy of the model using K-Nearest Neighbor and using streamlit to generate GUI.

Both of each member work together to find the suitable and interesting topic on Assignment. Anis managed to find the dataset and original code of the Phishing URL Detection on Kaggle. Then, we managed to divide the content of this assignment where Anis did the *Introduction* and *AI solution/technique* and Sorfina did the *Conclusion* and *Result*.

Based on this Assignment, we learnt that it is not easy to know whether the link is a phishing link or not. Since all of us mostly click many links per day, somehow without realizing we might have clicked the wrong one. There are many ways to avoid phishing attacks such as when you accidentally click suspicious links, do not enter any personal information such as banking or sensitive data. We always need to be careful when we want to enter personal/sensitive information on any website. Moreover, we must always use HTTPS instead of HTTP, since it is secure and encrypted. HTTPS provides authentication of the website's identity to assure users that they are connecting to the legitimate website. However, phishing these days is advanced and clever, realizing that they need to be using HTTPS instead of HTTP. The most important one is to avoid clicking links that are not familiar and given out by strangers because we do not know the real intention of them.

Based on this assignment, we can observe the pattern of the phishing URL based on 30 features so that we can conclude how the phishing URL can be seen. It is hard to recognize phishing URLs, however, based on this project, we can see how the URL can be classified into phishing or legitimate URL. After the program being trained using the datasets, the program can recognize the phishing URL based on 30 features like URL's having "@" symbol, Redirecting using "/", Favicon, Domain Registration Length, Using Non-Standard Port, Url of Anchor and more. Based on these features, the program can detect anomalies in the URL that can be considered as a phishing URL by training and to get

a better result, the program should be training for a long time to ensure false positive reduction. This can refine their understanding of legitimate URLs and their ability to distinguish between legitimate and phishing URLs.

The improvement that we can make throughout this assignment is implementing behavioral analysis which is using techniques to detect phishing URLs based on user behavior. Using this strategy, users' interactions with emails, messages, and webpages are observed and understood. Phishing attacks frequently rely on deceiving users into clicking on dangerous links or divulging personal data. It is feasible to indicate potentially hazardous URLs by looking for patterns in user activity, such as odd click patterns or interactions with suspicious emails. For instance, it can be a sign of phishing if a person clicks a link in an email and instantly enters sensitive data on the ensuing website. Phishing URL detection systems can evolve and enhance their capacity to detect possible threats in real-time by routinely observing and analyzing user behavior.

7. Reference

1. Abdul Samad, S. R., Balasubaramanian, S., Al-Kaabi, A. S., Sharma, B., Chowdhury, S., Mehbodniya, A., ... & Bostani, A. (2023). Analysis of the Performance Impact of Fine-Tuned Machine Learning Model for Phishing URL Detection. *Electronics*, 12(7), 1642.
2. Altaher, A. (2017). Phishing websites classification using hybrid SVM and KNN approach. *International Journal of Advanced Computer Science and Applications*, 8(6).
3. Altaher, A. (2017). Phishing websites classification using hybrid SVM and KNN approach. *International Journal of Advanced Computer Science and Applications*, 8(6).
4. Assegie, T. A. (2021). K-nearest neighbor based URL identification model for phishing attack detection. *Indian Journal of Artificial Intelligence and Neural Networking*, 1, 18-21.
5. Chand, E. (2020, February 28). Phishing website detector. Kaggle.
<https://www.kaggle.com/datasets/eswarchandt/phishing-website-detector>
6. Chand, E.(2020, February 28). Website phishing. Kaggle.
<https://www.kaggle.com/code/eswarchandt/website-phishing/notebook>
7. Das Gupta, S., Shahriar, K.T., Alqahtani, H. et al. Modeling Hybrid Feature-Based Phishing Websites Detection Using Machine Learning Techniques. *Ann. Data. Sci.* (2022).
<https://doi.org/10.1007/s40745-022-00379-8>
8. Dutta, A. K. (2021). Detecting phishing websites using machine learning technique. *PloS one*, 16(10), e0258361.
9. James, J., Sandhya, L., & Thomas, C. (2013, December). Detection of phishing URLs using machine learning techniques. In *2013 International conference on control communication and computing (ICCC)* (pp. 304-309). IEEE
10. Mohammad, R. M., Thabtah, F., & McCluskey, L. (2015). Phishing websites features. *School of Computing and Engineering, University of Huddersfield*.
11. PhishTank. (n.d.). Join the fight against phishing. <https://phishtank.org/>
12. R, S. E. (2021, June). Understand random forest algorithms with examples . Analytics Vidhya.
<https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/>
13. Ram S. (2023, April). Phishingdetection_with_input_as_URL. Kaggle.
<https://www.kaggle.com/code/santhoshram19/phishingdetection-with-input-as-url>
14. Safi, A., & Singh, S. (2023). A Systematic Literature Review on Phishing Website Detection Techniques. *Journal of King Saud University-Computer and Information Sciences*.
15. Shahrivari, V., Darabi, M. M., & Izadi, M. (2020). Phishing detection using machine learning

techniques. *arXiv preprint arXiv:2009.11116*.

16. Simister, A. (2022, November 24). 10 ways to prevent phishing attacks. Lepide Blog: A Guide to IT Security, Compliance and IT Operations.
<https://www.lepide.com/blog/10-ways-to-prevent-phishing-attacks/>
17. Xun Dong, Clark, J. A., & Jacob, J. L. (2008). User behaviour based phishing websites detection. 2008 International Multiconference on Computer Science and Information Technology. <https://doi.org/10.1109/imcsit.2008.4747332>